

HW2.5 — GPU Assignment I: Precision, Bandwidth, and the Cost of Attention

Naga Sai Haritha Gottumukkala

Department of Applied Data Science

San Jose State University

San Jose, California, USA

DATA 266 — Generative AI and Large Language Models

SID4: 1598 SEED: 1598

Abstract

This assignment gave me a chance to study GPU performance by measuring it directly instead of only reading numbers from a specification sheet. I completed the experiments on an NVIDIA GeForce RTX 4090 workstation. The main goal was to understand two hardware limits that affect most deep-learning workloads: how quickly the GPU can perform matrix multiplication and how quickly it can move data through memory.

I first recorded the GPU identity, software environment, VRAM, power limit, and vendor specifications. I then benchmarked dense matrix multiplication in FP32, TF32, FP16, and BF16 at matrix sizes 1024, 4096, 8192, and 16384. The best required BF16 result was 164.69 TFLOPS at $N=8192$, which was 99.69% of the dense 165.2 TFLOPS reference used for the comparison. Because the installed software stack also exposed FP8, I ran an additional FP8 E4M3 experiment.

The second part of the work compared a memory-bound elementwise operation with a compute-bound matrix multiplication. The elementwise test reached 920.74 GB/s, or 91.34% of the RTX 4090's specified 1008 GB/s memory bandwidth. Arithmetic-intensity calculations helped me understand why the two operations fall on different sides of the roofline.

The attention experiments were the most useful part of the assignment for understanding Transformers. I implemented scaled dot-product attention naively and measured memory and latency as sequence length increased. At sequence length 16,384, the naive implementation reported about 32.26 GiB peak allocated memory and 892.22 ms median forward latency. The measured memory curve followed a clear quadratic relationship. I then tested optimized attention. My first forced Flash Attention attempt failed with “No available kernel,” so I investigated the available PyTorch backends and repeated the experiment using the working Efficient Attention backend.

Finally, I ran a sustained BF16 compute workload for approximately 20 minutes while recording clock frequency, temperature, power, utilization, and memory usage. The final five-minute average throughput was 160.11 TFLOPS compared with a 168.96 TFLOPS peak during the first 30 seconds, giving a steady-state-to-peak ratio of 94.76%. The maximum recorded temperature was 77 degrees Celsius, and no clear severe thermal-throttle onset was observed.

Overall, this assignment helped me connect GPU specifications with the performance that I actually measured on a real card.

Index Terms—GPU benchmarking, RTX 4090, CUDA, PyTorch, FP32, TF32, FP16, BF16, FP8, memory bandwidth, arithmetic intensity, roofline model, scaled dot-product attention, memory-efficient attention, thermal behavior

I. INTRODUCTION

Before this assignment, I mostly looked at GPU performance through specifications such as TFLOPS, VRAM capacity, and memory bandwidth. Those values are useful, but they do not explain how the GPU will behave for every real workload.

This assignment was designed to make that difference visible before attention and Transformers are used more heavily later in the course. Two limits matter repeatedly in those workloads. The first is how quickly the GPU can

perform matrix multiplication. The second is how quickly data can move through GPU memory.

A workload can be limited by either one. For example, a large matrix multiplication performs a great deal of arithmetic and can become compute-bound. A simple elementwise operation performs very little arithmetic for each value that has to be read or written, so memory bandwidth can become the main limit.

The attention experiment connects these hardware ideas directly to Transformers. In a direct implementation of scaled dot-product attention, multiplying Query by Key

transpose creates an $L \times L$ matrix, where L is the sequence length. As L grows, this intermediate matrix grows very quickly.

Instead of only accepting that attention has quadratic memory cost, I wanted to see the behavior in measurements from the RTX 4090 assigned to me.

The last part of the assignment also looked beyond short benchmarks. A GPU can behave differently after it has been running near full load for many minutes. I therefore ran a sustained compute test and recorded temperature, clocks, power, and utilization throughout the run.

The rest of this report follows the assignment in order. Section II describes the experimental environment and Part A provenance. Section III presents the precision benchmarks from Part B. Section IV covers the roofline experiment from Part C. Section V presents the attention experiments from Part D. Section VI discusses the sustained-load experiment from Part E. Section VII contains the required Part F summary table, followed by failure analysis, reproducibility, limitations, and my final observations.

II. EXPERIMENTAL SETUP AND PART A PROVENANCE

A. Standing Course Parameters

I kept the standing DATA 266 parameters with the experiment so that the repository follows the same reproducibility setup used in the other assignments.

TABLE I. STANDING DATA 266 PARAMETERS

Parameter	Value
SID4	1598
SEED	1598
SLICE	598
HP_ID	2
CLS_A	8
CLS_B	5

The benchmark scripts use the same seed where random tensors are generated. Exact benchmark timing is not expected to reproduce bit-for-bit because GPU clocks, temperature, background activity, driver state, and power conditions can change slightly between runs.

B. GPU Reservation and Usage

I used Machine 2 in SJSU room ISB 836. The assigned GPU was an NVIDIA GeForce RTX 4090.

The reservation began on September 10, 2026 at 12:00 PM PDT and ended on September 11, 2026 at 11:59 AM PDT.

I kept the reservation duration separate from the GPU time that I could directly support using retained timestamps. The earliest saved GPU evidence was recorded at approximately 15:04:30 PDT on September 10, and the final sustained workload completed at approximately 17:33:03 PDT.

The minimum directly evidenced usage was therefore approximately

$$2\text{ h }28\text{ min }33\text{ s} \approx 2.48\text{ GPU hours} \quad (1)$$

The reservation information and GPU-hour record are also preserved in GPU_HOURS.md and the reservation files in the repository.

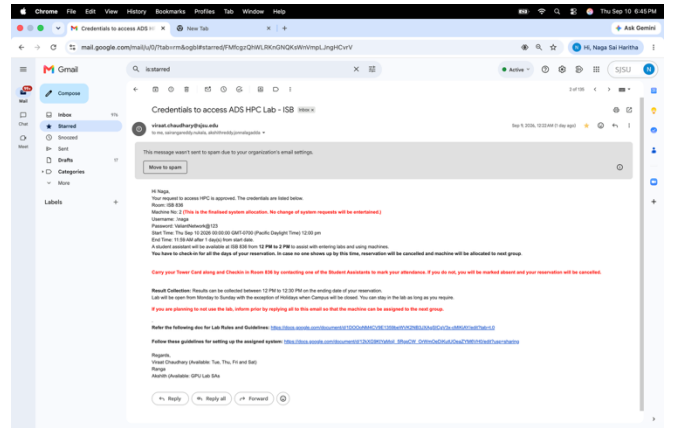


Fig. 1. Redacted reservation evidence for the assigned RTX 4090 workstation. Login credentials are intentionally not included.

C. Initial GPU Verification

Before running a benchmark, I first checked which GPU was actually assigned to the machine. This was important because every measurement in Parts B through E had to be traceable to the GPU that produced it.

I started with nvidia-smi. The first output confirmed that the machine contained an NVIDIA GeForce RTX 4090. I then collected the complete nvidia-smi -q output instead of relying only on the short summary.

Figs. 2 and 3 show the evidence I saved during this step.

PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> nvidia-smi | Tee-Object -FilePath raw\gpu_info\nvidia_smi_summary_4890.txt Thu Sep 10 15:04:30 2026

NVIDIA-SMI 610.60		Driver Version: 610.60		CUDA Version: 13.3			
GPU	Name	Perf	Driver-Model	Bus-Id	Disp.A		
Fan	Temp	Perf	Perf.Usage/Cap	Memory-Usage	Volatile Uncorr. ECC		
					Compute M. MIG M.		
0	NVIDIA GeForce RTX 4090	39C	P8	14M / 450M	108771B / 24564M1B	0%	OFF
						Default	N/A

Processes:							
GPU	CI	CI	PID	Type	Process name	GPU Memory Usage	
ID	ID	ID	ID	ID	ID	ID	
0	N/A	N/A	28268	C#G	...Bbwe\PhoneExperienceHost.exe	N/A	
0	N/A	N/A	21844	C#G	...xyewy\ShellExperienceHost.exe	N/A	
0	N/A	N/A	26880	C#G	...d.4191.66\usedgawebview2.exe	N/A	
0	N/A	N/A	26492	C#G	...Chrome\Application\chrome.exe	N/A	
0	N/A	N/A	27776	C#G	...Chrome\Application\chrome.exe	N/A	
0	N/A	N/A	31512	C#G	...r\Frontend\docker Desktop.exe	N/A	
0	N/A	N/A	36916	C#G	C:\Windows\explorer.exe	N/A	
0	N/A	N/A	35864	C#G	...crossoft OneDrive\OneDrive.exe	N/A	
0	N/A	N/A	36796	C#G	...yStartMenuExperienceHost.exe	N/A	
0	N/A	N/A	37868	C#G	...c6d61h2xyewy\SearchHost.exe	N/A	
0	N/A	N/A	41328	C#G	...App_c6d61h2xyewy\LockApp.exe	N/A	
0	N/A	N/A	41908	C#G	...e_x64_htrsf667h5m2\AMCC.exe	N/A	
0	N/A	N/A	47832	C#G	...t\Edge\Application\msedge.exe	N/A	
0	N/A	N/A	48040	C#G	...y868Bbwe\WindowsTerminal.exe	N/A	
0	N/A	N/A	48820	C#G	...SnippingTool\SnippingTool.exe	N/A	

PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5>

Fig. 2. Initial nvidia-smi check confirming the RTX 4090.

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> nvidia-smi -q | Tee-Object -FilePath raw\gpu_info\nvidia_smi_q_4890.txt
Thu Sep 10 15:04:30 2026
```

NVIDIA-SMI 610.60		Driver Version: 610.60		CUDA Version: 13.3	
GPU	Name	Perf	Driver-Model	Bus-Id	Disp.A
Fan	Temp	Perf	Perf.Usage/Cap	Memory-Usage	Volatile Uncorr. ECC
0	NVIDIA GeForce RTX 4090	39C	P8	14M / 450M	108771B / 24564M1B

Processes	GPU	CI	CI	PID	Type	Process name	GPU Memory Usage
ID	ID	ID	ID	ID	ID	ID	ID
0	N/A	N/A	N/A	28268	C#G	...Bbwe\PhoneExperienceHost.exe	N/A
0	N/A	N/A	N/A	21844	C#G	...xyewy\ShellExperienceHost.exe	N/A
0	N/A	N/A	N/A	26880	C#G	...d.4191.66\usedgawebview2.exe	N/A
0	N/A	N/A	N/A	26492	C#G	...Chrome\Application\chrome.exe	N/A
0	N/A	N/A	N/A	27776	C#G	...Chrome\Application\chrome.exe	N/A
0	N/A	N/A	N/A	31512	C#G	...r\Frontend\docker Desktop.exe	N/A
0	N/A	N/A	N/A	36916	C#G	C:\Windows\explorer.exe	N/A
0	N/A	N/A	N/A	35864	C#G	...crossoft OneDrive\OneDrive.exe	N/A
0	N/A	N/A	N/A	36796	C#G	...yStartMenuExperienceHost.exe	N/A
0	N/A	N/A	N/A	37868	C#G	...c6d61h2xyewy\SearchHost.exe	N/A
0	N/A	N/A	N/A	41328	C#G	...App_c6d61h2xyewy\LockApp.exe	N/A
0	N/A	N/A	N/A	41908	C#G	...e_x64_htrsf667h5m2\AMCC.exe	N/A
0	N/A	N/A	N/A	47832	C#G	...t\Edge\Application\msedge.exe	N/A
0	N/A	N/A	N/A	48040	C#G	...y868Bbwe\WindowsTerminal.exe	N/A
0	N/A	N/A	N/A	48820	C#G	...SnippingTool\SnippingTool.exe	N/A

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5>
```

Fig. 2 (cont.). Complete nvidia-smi -q capture used for provenance.

The GPU UUID recorded for the experiments was

GPU-b60240a9-31a3-126f-751f-afc1dcc1d73f

The UUID matters because it gives a more reliable identity than only writing “RTX 4090.” The same UUID was included with the important run records later in the assignment.

I also checked PyTorch and CUDA before beginning the expensive experiments.

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> nvidia-smi -query --format={name,uuid,driver_version,memory.total,power.limit} --noheader | Tee-Object -FilePath raw\gpu_info\gpu_identity_4890.txt
name,uuid,driver_version,memory.total,power.limit
NVIDIA GeForce RTX 4090, GPU-b60240a9-31a3-126f-751f-afc1dcc1d73f, 610.60, 24564 MiB, 450.00 W
```

Fig. 3. GPU identity query showing the UUID and hardware information.

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> python src\gpu_preflight.py | Tee-Object -FilePath raw\gpu_info\preflight_4890.txt
Python version : 3.13.2
Operating system : Windows-11-10.0.22631-SP0
PyTorch version : 2.11.0+cu128
PyTorch CUDA runtime : 12.8
CUDA available : True
CUDA device index : 0
GPU name : NVIDIA GeForce RTX 4090
Compute capability : 8.9
Total VRAM : 23.99 GiB
BF16 supported : True
Float32 / TF32 CONFIGURATION
Float32 matmul precision: highest
TF32 matmul allowed: False
FP32 MATRIX MULTIPLICATION TEST
FP32 matrix multiplication: SUCCESS
FP16 MATRIX MULTIPLICATION TEST
FP16 matrix multiplication: SUCCESS
BF16 MATRIX MULTIPLICATION TEST
BF16 matrix multiplication: SUCCESS
GPU MEMORY STATUS
Allocated memory : 8.12 MiB
Reserved memory : 20.80 MiB
PRE-FLIGHT CHECK COMPLETE
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5>
```

Fig. 3 (cont.). PyTorch/CUDA check confirming CUDA availability and BF16 support.

D. CUDA Toolkit and Preflight Check

The CUDA version reported by the NVIDIA driver is not necessarily the same value as the CUDA toolkit used to

compile or run PyTorch code. For this reason, I recorded both instead of trying to make them appear identical.

The installed PyTorch build was 2.11.0+cu128, and the PyTorch CUDA runtime reported version 12.8. The CUDA toolkit check through nvcc also reported CUDA 12.8.

Before starting the main measurements, I ran the preflight script one more time to check that the environment was ready.

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> nvcc --version 2>61 | Tee-Object -FilePath raw\gpu_info\nvcc_4890.txt
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Fri_Feb_21_20:42:46_Pacific_Standard_Time_2025
Cuda compilation tools, release 12.8, V12.8.93
Build cuda_12.8.r12.8/compiler.35583870_0
```

Fig. 4. nvcc output used to record the installed CUDA toolkit.

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> python src\gpu_preflight.py | Tee-Object -FilePath raw\gpu_info\preflight_4890.txt
Python version : 3.13.2
Operating system : Windows-11-10.0.22631-SP0
PyTorch version : 2.11.0+cu128
PyTorch CUDA runtime : 12.8
CUDA available : True
CUDA device index : 0
GPU name : NVIDIA GeForce RTX 4090
Compute capability : 8.9
Total VRAM : 23.99 GiB
BF16 supported : True
Float32 / TF32 CONFIGURATION
Float32 matmul precision: highest
TF32 matmul allowed: False
FP32 MATRIX MULTIPLICATION TEST
FP32 matrix multiplication: SUCCESS
FP16 MATRIX MULTIPLICATION TEST
FP16 matrix multiplication: SUCCESS
BF16 MATRIX MULTIPLICATION TEST
BF16 matrix multiplication: SUCCESS
GPU MEMORY STATUS
Allocated memory : 8.12 MiB
Reserved memory : 20.80 MiB
PRE-FLIGHT CHECK COMPLETE
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5>
```

Fig. 4 (cont.). Final GPU preflight check before the measured experiments.

After completing the provenance checks, I also verified that the Part A files had been saved in the repository.

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> dir raw\gpu_info
Directory: C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5\raw\gpu_info
```

Mode	LastWriteTime	Length	Name
-a----	9/10/2026 12:44 PM	0	.gitkeep
-a----	9/10/2026 3:07 PM	324	gpu_identity_4890.txt
-a----	9/10/2026 3:11 PM	464	nvcc_4890.txt
-a----	9/10/2026 3:05 PM	25793	nvidia_smi_4890.txt
-a----	9/10/2026 3:04 PM	6024	nvidia_smi_summary_4890.txt
-a----	9/10/2026 3:19 PM	3914	preflight_4890.txt
-a----	9/10/2026 3:09 PM	210	pytorch_cuda_4890.txt

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> dir raw\screenshots\4890
Directory: C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5\raw\screenshots\4890
```

Mode	LastWriteTime	Length	Name
-a----	9/10/2026 3:05 PM	94911	A1_nvidia_smi_4890.png
-a----	9/10/2026 3:07 PM	113778	A2_nvidia_smi_q_4890.png
-a----	9/10/2026 3:08 PM	19293	A3_gpu_identity_uuid_4890.png
-a----	9/10/2026 3:10 PM	33316	A4_pytorch_cuda_4890.png
-a----	9/10/2026 3:12 PM	22974	A5_nvcc_4890.png
-a----	9/10/2026 3:21 PM	73274	A6_preflight_4890.png
-a----	9/10/2026 2:58 PM	80533	nvidia-smi.png

Fig. 5. Part A output files saved before moving to the performance benchmarks.

PS C:\Users\naga\Desktop\DATA266-1598_hwl\hw2_5> nvidia-smi									
Thu Sep 10 14:55:59 2026									
NVIDIA-SMI 610.60			KMD Version: 610.60			CUDA UMD Version: 13.3			
GPU Name		Driver-Model		Bus-Id	Disp.A	Volatile	Uncorr.	ECC	
Fan	Temp	Perf	Pwr:Usage/Cap						
					Memory-Usage	GPU-Util	Compute	M.	
							MIG	M.	
0	NVIDIA	GeForce RTX 4090	WDDM	00000000:01:00:00	On			Off	
0%	38C	P8	14W / 450W	840M1B / 24564M1B		1%	Default	N/A	
Processes:									
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage			
ID	ID	ID							
0	N/A	N/A	28268	C+G	...8bbwe\PhoneExperienceHost.exe	N/A			
0	N/A	N/A	21844	C+G	...xyem\ShellExperienceHost.exe	N/A			
0	N/A	N/A	26900	C+G	...0_4191_66\msedgewebview2.exe	N/A			
0	N/A	N/A	26492	C+G	...Chrome\Application\chrome.exe	N/A			
0	N/A	N/A	27776	C+G	...Chrome\Application\chrome.exe	N/A			
0	N/A	N/A	31512	C+G	...F\Frontend\Docker Desktop.exe	N/A			
0	N/A	N/A	34916	C+G	C:\Windows\explorer.exe	N/A			
0	N/A	N/A	35864	C+G	...croscof OneDrive\OneDrive.exe	N/A			
0	N/A	N/A	36976	C+G	...Y\StartMenuExperienceHost.exe	N/A			
0	N/A	N/A	37000	C+G	...CmNhl32xyem\SearchHost.exe	N/A			
0	N/A	N/A	41324	C+G	...App_CmNhl32xyem\LockApp.exe	N/A			
0	N/A	N/A	41900	C+G	...0_x64_https6675hkn2\AVMCC.exe	N/A			
0	N/A	N/A	47832	C+G	...tEdge\Application\msedge.exe	N/A			
0	N/A	N/A	48440	C+G	...yb3d8bbwe\WindowsTerminal.exe	N/A			

[illegible]

E. Measured Environment

TABLE II. RTX 4090 EXPERIMENTAL ENVIRONMENT

F. RTX 4090 Vendor Specifications

buffer, a 384-bit memory interface, and a specified memory bandwidth of 1008 GB/s for the RTX 4090.

Ada uses fourth-generation Tensor Cores. The architecture supports reduced-precision Tensor Core modes relevant to this assignment, including TF32, FP16, BF16, and FP8 [1].

Property	RTX 4090
Architecture	NVIDIA Ada Lovelace
Memory type	GDDR6X
Frame buffer	24 GB
Memory interface	384-bit
Specified bandwidth	1008 GB/s
Tensor Core generation	4th Generation
Reduced precisions	TF32, FP16, BF16, FP8 and integer modes

For the dense theoretical-throughput comparisons in Part B, I used the reference values shown in Table IV. These are dense references; I did not use sparse-performance values because my matrix benchmark did not use structured sparsity.

Precision	Reference
FP32	82.6 TFLOPS
TF32 Tensor	82.6 TFLOPS
FP16 Tensor, FP32 accumulate	165.2 TFLOPS
BF16 Tensor, FP32 accumulate	165.2 TFLOPS

III. PART B: PRECISION AND ACHIEVED THROUGHPUT

A. What I Was Measuring

The purpose of this part was to see how matrix size and numerical precision affect the throughput I actually receive from the GPU.

I tested four required precision modes: FP32, TF32, FP16, and BF16. FP32 served as the full-precision baseline. TF32 allows compatible matrix multiplication to use the Tensor Core path while keeping FP32 input tensors. FP16 and BF16 use 16-bit formats and can provide much higher Tensor Core throughput.

I kept the explanation short because the main goal was not to compare floating-point formats theoretically. I wanted to see how they behaved on the assigned card.

B. Benchmark Design

The required matrix sizes were

$$N \in \{1024, 4096, 8192, 16384\} \quad (2)$$

Each required precision/size configuration used five warm-up operations followed by 20 timed repetitions.

I warmed the GPU first because the first matrix multiplication can contain startup effects that do not represent the later repetitions. I also synchronized CUDA around the measured work and used the median runtime.

For a dense square matrix multiplication,

$$C = AB \quad (3)$$

the approximate arithmetic work is

$$F = 2N^3 \quad (4)$$

If the median runtime is t seconds, achieved throughput is

$$P = 2N^3 / (t \times 10^{12}) \text{ TFLOPS} \quad (5)$$

The percentage of the theoretical reference is

$$\eta = (P_{\text{measured}} / P_{\text{theoretical}}) \times 100 \quad (6)$$

The UUID-labelled run for these measurements was B-4090-MATMUL.

C. Initial Validation and Full Run

Before using GPU time on the complete experiment, I first created the benchmark script and ran a smaller validation. Once the quick test completed without an error, I moved to the full precision and matrix size sweep.

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> dir src

Directory: C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5\src

Mode                LastWriteTime         Length Name
----                -
-a----          9/10/2026   3:24 PM             7286 benchmark_matmul.py
-a----          9/10/2026  12:44 PM             3999 capture_gpu_info.sh
-a----          9/10/2026  12:44 PM             4151 gpu_preFlight.py
```

Fig. 7. Matrix benchmark script prepared for the RTX 4090 run.

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> python src\benchmark_matmul.py --precision FP32 --size 1024 --warmup 5 --reps 20 --test-object cuda --matmul-version cuda
BENCHMARK COMPLETE
Results saved to: src\results\matmul_4090.csv
Matmul configuration: 1024
```

Fig. 7 (cont.). Quick validation run before the full precision sweep.

The complete required run and the saved result table are shown in Fig. 8.

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> python src\benchmark_matmul.py --precision FP32,TF32 --size 1024,4096,8192,16384 --warmup 5 --reps 20 --test-object cuda --matmul-version cuda
BENCHMARK COMPLETE
Results saved to: src\results\matmul_4090.csv
Matmul configuration: 1024
```

Fig. 8. Full matrix benchmark covering all required sizes and precisions.

N	precision	median_ms	stdev_ms	achieved_tflops	status
1024	FP32	0.057308970504289	0.003779021114104	37.4493236382196	SUCCESS
1024	TF32	0.05227968781916	0.003626531310407	35.1381628179296	SUCCESS
4096	FP32	2.484706861381	0.2086604876465196	54.9827187782513	SUCCESS
4096	TF32	1.67312556111	0.18181852128018	66.7898262733855	SUCCESS
8192	FP32	19.9961186861381	0.001368462656112379	41.136281866461996	SUCCESS
8192	TF32	12.4943106664619	0.112699162807076	82.1465311971819	SUCCESS
16384	FP32	173.186918218683	0.218366412196865	88.0888642377121	SUCCESS
16384	TF32	101.45641218683	0.112699162807076	104.628216861644	SUCCESS
1024	FP16	0.057308970504289	0.003626531310407	76.271862818621	SUCCESS
1024	TF16	0.05227968781916	0.112699162807076	149.386812172946	SUCCESS
4096	FP16	0.057308970504289	0.112699162807076	149.386812172946	SUCCESS
4096	TF16	0.05227968781916	0.112699162807076	149.386812172946	SUCCESS
8192	FP16	0.057308970504289	0.112699162807076	149.386812172946	SUCCESS
8192	TF16	0.05227968781916	0.112699162807076	149.386812172946	SUCCESS
16384	FP16	0.057308970504289	0.112699162807076	149.386812172946	SUCCESS
16384	TF16	0.05227968781916	0.112699162807076	149.386812172946	SUCCESS

Fig. 8 (cont.). Saved matrix results displayed after the complete run.

D. Matrix Multiplication Results

Table V contains the required measured values.

TABLE V. DENSE MATRIX MULTIPLICATION RESULTS ON THE RTX 4090

N	Precision	Median (ms)	Achieved TFLOPS	Theory TFLOPS	% Theory
1024	FP32	0.0573	37.45	82.6	45.34%
4096	FP32	2.4847	55.31	82.6	66.97%
8192	FP32	19.9961	54.99	82.6	66.57%
16384	FP32	173.1869	50.79	82.6	61.49%
1024	TF32	0.0522	41.12	82.6	49.78%
4096	TF32	1.6731	82.14	82.6	99.45%
8192	TF32	12.4943	88.00	82.6	106.54%
16384	TF32	101.4564	86.70	82.6	104.96%

N	Precision	Median (ms)	Achieved TFLOPS	Theory TFLOPS	% Theory
1024	FP16	0.0306	70.27	165.2	42.54%
4096	FP16	0.8115	169.36	165.2	102.52%
8192	FP16	6.7781	162.22	165.2	98.19%
16384	FP16	55.9340	157.26	165.2	95.19%
1024	BF16	0.0266	80.66	165.2	48.83%
4096	BF16	0.8787	156.42	165.2	94.68%
8192	BF16	6.6764	164.69 (best)	165.2	99.69% (best)
16384	BF16	54.8971	160.23	165.2	96.99%

The highest required BF16 result was 164.69 TFLOPS at N=8192. Using the dense 165.2 TFLOPS reference,

$$(164.6871 / 165.2) \times 100 = 99.69\% \quad (7)$$

This was the point where BF16 came closest to the reference value used in my report.

E. Plot Generation

After saving the CSV data, I checked Matplotlib, created the plotting script, and generated the throughput figure from the saved measurements.

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> python -c "import matplotlib; print('matplotlib:', matplotlib.__version__)" | Tee-Object -File obj\src\matmul\matplotlib_check_4090.txt
matplotlib: 3.11.1
```

Fig. 9. Matplotlib check before generating the result figure.

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> Get-Item src\plot_matmul.py

Directory: C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5\src

Mode                LastWriteTime         Length Name
----                -
-a-----          9/10/2026   3:37 PM           1328 plot_matmul.py
```

Fig. 9 (cont.). Plotting script prepared using the saved benchmark CSV.

The plot command completed successfully as shown in Fig. 10.

```
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5> python src\plot_matmul.py | Tee-Object -File obj\src\matmul\plot_matmul_4090.txt
Plot created successfully.
Saved: figures\matmul_throughput_4090.png
FP32: peak 55.314 TFLOPS at N=4096
TF32: peak 88.001 TFLOPS at N=8192
FP16: peak 169.360 TFLOPS at N=4096
BF16: peak 164.687 TFLOPS at N=8192
PS C:\Users\naga\Desktop\DATA266-1598_hw1\hw2_5>
```

Fig. 10. Terminal output confirming that the matrix-throughput plot was generated.

The clean scientific plot is shown in Fig. 11.

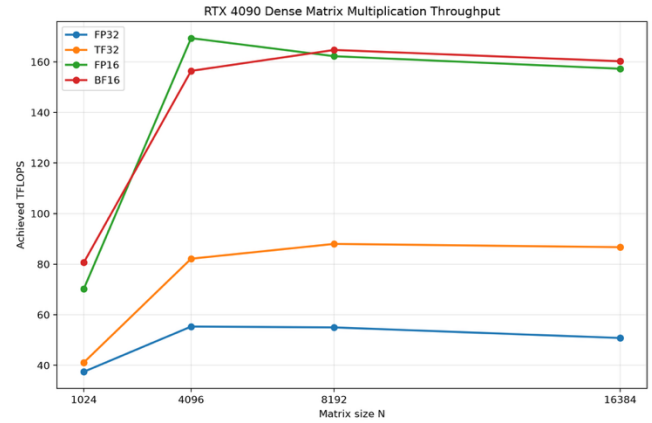


Fig. 11. Achieved matrix multiplication throughput versus matrix size. Each line represents one required numerical precision.

The repository also contains the retained Part B plot artifact with the assignment-style filename.

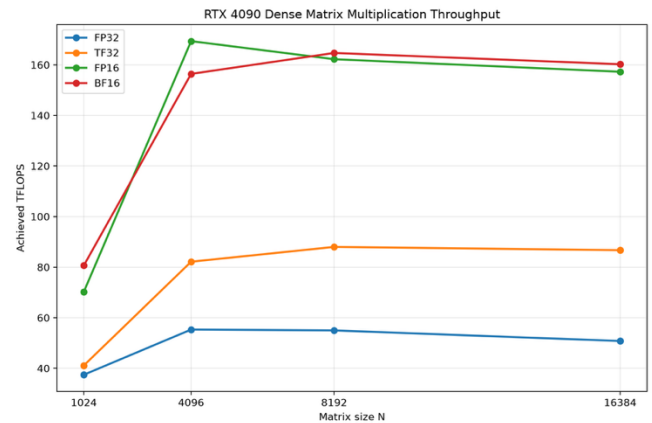


Fig. 12. Retained Part B throughput plot artifact saved with the benchmark evidence.

F. Where the Throughput Plateau Appeared

The smallest matrix did not keep the RTX 4090 busy long enough to reach its strongest throughput. At N=1024,

every required precision was well below its later high-performance region.

FP32 reached its main plateau around N=4096. The measured throughput was 55.31 TFLOPS at 4096 and 54.99 TFLOPS at 8192, which are almost the same.

TF32 entered its high-throughput region around N=4096 and reached 88.00 TFLOPS at N=8192.

FP16 produced its highest measured value at N=4096, where it reached 169.36 TFLOPS. Increasing the matrix again did not increase the measured TFLOPS.

BF16 increased from 156.42 TFLOPS at N=4096 to its best required result of 164.69 TFLOPS at N=8192. Increasing to 16,384 reduced it slightly to 160.23 TFLOPS. I therefore treated 8192 as the clearest BF16 plateau point in my run.

The main reason the small matrix never reaches the same level is that the GPU needs enough parallel work to keep its resources occupied. Kernel launch and setup overhead also form a larger percentage of a very short operation.

G. Measurements Slightly Above 100 Percent

A few results were slightly above the fixed theoretical references. For example, TF32 reached 106.54% at N=8192, and FP16 reached 102.52% at N=4096.

I did not manually cap these values.

The theoretical references are fixed nominal values, while a consumer GPU can operate at workload-dependent boost clocks. Short timing measurements also contain normal experimental variation. I therefore kept the measured values and documented the difference instead of editing the data to match an expected limit.

H. Additional FP8 Experiment

The assignment asked me to run a lower precision if the installed software stack exposed one.

I first checked whether PyTorch exposed FP8 support rather than assuming that the hardware capability automatically meant the API was usable.

```
PS C:\Users\naga\desktop\DATA266-1598_hw1\hw2_5> python src/test_fp8.py | Tee-Object -FilePath raw\matmul\fp8_probe_4090.txt
HW2.5 PART B4 à LOWER PRECISION / FP8 PROBE
=====
PyTorch: 2.11.0+cu128
CUDA runtime: 12.8
CUDA available: True
GPU: NVIDIA GeForce RTX 4090
GPU UUID: GPU-b60240a9-31a3-126f-751f-afcdccid73f
Compute capability: 8.9

FP8 API availability:
torch.float8_e4m3fn: True
torch.float8_e4m3: True
torch_scaled_mm: True
torch.nn.functional_scaled_mm: True

Attempting FP8 E4M3 matrix multiplication...
API used: torch_scaled_mm
Output shape: (1024, 1024)
Output dtype: torch.bfloat16
FP8 E4M3 GEMM: SUCCESS

=====
FP8 PROBE COMPLETE
=====
PS C:\Users\naga\desktop\DATA266-1598_hw1\hw2_5>
```

Fig. 13. FP8 datatype and operation capability check.

```
PS C:\Users\naga\desktop\DATA266-1598_hw1\hw2_5> python src/benchmark_fp8.py | Tee-Object -FilePath raw\matmul\fp8_benchmark_run_4090.txt
HW2.5 PART B4 à FP8 E4M3 THROUGHPUT BENCHMARK
=====
Timestamp: 2025-09-10T15:56:01.433055
GPU: NVIDIA GeForce RTX 4090
GPU UUID: GPU-b60240a9-31a3-126f-751f-afcdccid73f
PyTorch: 2.11.0+cu128
CUDA runtime: 12.8
Input precision: FP8 E4M3
Output dtype: BF16
Warmup iterations: 5
Timed repetitions: 20
Quantization included in timing: NO

Running FP8 E4M3, N=1024 ... 0.0440 ms | 48.869 TFLOPS
Running FP8 E4M3, N=4096 ... 0.4669 ms | 294.337 TFLOPS
Running FP8 E4M3, N=8192 ... 3.4447 ms | 319.193 TFLOPS
Running FP8 E4M3, N=16384 ... 26.1871 ms | 335.894 TFLOPS

=====
FP8 BENCHMARK COMPLETE
=====
Saved: raw\matmul\fp8_benchmark_4090.csv
```

Fig. 13 (cont.). Initial FP8 scaled-matrix-multiplication probe.

The probe succeeded, as shown in Fig. 14, so I moved to the complete FP8 E4M3 benchmark.

```
PS C:\Users\naga\desktop\DATA266-1598_hw1\hw2_5> Get-Content raw\matmul\fp8_probe_4090.txt
HW2.5 PART B4 à LOWER PRECISION / FP8 PROBE
=====
PyTorch: 2.11.0+cu128
CUDA runtime: 12.8
CUDA available: True
GPU: NVIDIA GeForce RTX 4090
GPU UUID: GPU-b60240a9-31a3-126f-751f-afcdccid73f
Compute capability: 8.9

FP8 API availability:
torch.float8_e4m3fn: True
torch.float8_e4m3: True
torch_scaled_mm: True
torch.nn.functional_scaled_mm: True

Attempting FP8 E4M3 matrix multiplication...
API used: torch_scaled_mm
Output shape: (1024, 1024)
Output dtype: torch.bfloat16
FP8 E4M3 GEMM: SUCCESS

=====
FP8 PROBE COMPLETE
=====
```

Fig. 14. Successful FP8 probe result.

```
PS C:\Users\naga\desktop\DATA266-1598_hw1\hw2_5> python src/benchmark_fp8.py | Tee-Object -FilePath raw\matmul\fp8_benchmark_run_4090.txt
HW2.5 PART B4 à FP8 E4M3 THROUGHPUT BENCHMARK
=====
Timestamp: 2025-09-10T15:56:01.433055
GPU: NVIDIA GeForce RTX 4090
GPU UUID: GPU-b60240a9-31a3-126f-751f-afcdccid73f
PyTorch: 2.11.0+cu128
CUDA runtime: 12.8
Input precision: FP8 E4M3
Output dtype: BF16
Warmup iterations: 5
Timed repetitions: 20
Quantization included in timing: NO

Running FP8 E4M3, N=1024 ... 0.0440 ms | 48.869 TFLOPS
Running FP8 E4M3, N=4096 ... 0.4669 ms | 294.337 TFLOPS
Running FP8 E4M3, N=8192 ... 3.4447 ms | 319.193 TFLOPS
Running FP8 E4M3, N=16384 ... 26.1871 ms | 335.894 TFLOPS

=====
FP8 BENCHMARK COMPLETE
=====
Saved: raw\matmul\fp8_benchmark_4090.csv
```

Fig. 14 (cont.). Full FP8 E4M3 matrix benchmark.

The retained FP8 result table is shown in Fig. 15, and the numerical values are summarized in Table VI.

```
PS C:\Users\naga\desktop\DATA266-1598_hw1\hw2_5> Import-Csv raw\matmul\fp8_benchmark_4090.csv |
>> Format-Table N,input_precision,median_ms,stdev_ms,achieved_tflops,status -AutoSize
N      input_precision median_ms stdev_ms achieved_tflops status
-----
1024   FP8_E4M3        0.04395199939608570 0.005417194470581601 48.85974876017244 SUCCESS
4096   FP8_E4M3        0.465940409096562805 0.00280921105380022 294.337117092519 SUCCESS
8192   FP8_E4M3        3.4446566014442444 0.08991648435997641 319.19344772132485 SUCCESS
16384  FP8_E4M3        26.187088012695312 0.1278112989611375 335.894278481632 SUCCESS
```

Fig. 15. Saved FP8 E4M3 benchmark results.

TABLE VI. FP8 E4M3 MATRIX MULTIPLICATION RESULTS

N	Median (ms)	Achieved TFLOPS
1024	0.0440	48.86
4096	0.4669	294.34
8192	3.4447	319.19
16384	26.1871	335.89 (best)

The strongest measured FP8 result was about 335.89 TFLOPS at N=16384.

This part also showed me that hardware support and software support are not exactly the same thing. The card may support a datatype architecturally, but the installed framework still needs to expose a usable operation.

IV. PART C: BANDWIDTH-BOUND VS. COMPUTE-BOUND

A. Why I Ran Two Different Operations

The matrix benchmark mainly measured compute throughput. Part C asked a different question: what happens when the GPU spends most of its time moving data instead of doing arithmetic?

To compare the two cases, I used a large FP32 elementwise addition as the memory-bound operation and a large square FP32 matrix multiplication as the compute-bound operation.

The UUID-labelled run ID was C-4090-ROOFLINE.

B. Memory-Bound Elementwise Addition

The memory-heavy operation was

$$C_i = A_i + B_i \quad (8)$$

For every FP32 output element, the simplified traffic is two 4-byte reads and one 4-byte write. That gives approximately 12 bytes moved for one floating-point addition.

The arithmetic intensity is therefore

$$I_{add} = 1/12 = 0.08333 \text{ FLOPs/byte} \quad (9)$$

I used 200,000,000 FP32 elements. The measured median runtime was about 2.6066 ms, which produced an effective bandwidth of

$$920.74 \text{ GB/s} \quad (10)$$

Relative to the RTX 4090's specified 1008 GB/s bandwidth,

$$(920.74 / 1008) \times 100 = 91.34\% \quad (11)$$

C. Compute-Bound Matrix Multiplication

For the compute-heavy case, I used an FP32 8192×8192 matrix multiplication.

Using the simple traffic assumption that the two input matrices are read and the output matrix is written once,

$$I_{matmul} = 2N^3 / (3N^2 \cdot 4) = N/6 \quad (12)$$

At N=8192,

$$I_{matmul} = 1365.33 \text{ FLOPs/byte} \quad (13)$$

D. Roofline Interpretation

The ridge point can be estimated from the FP32 compute reference and the specified memory bandwidth [3]:

$$I_{ridge} = (82.6 \times 10^{12}) / (1008 \times 10^9) = 81.94 \text{ FLOPs/byte} \quad (14)$$

The elementwise addition has an intensity of only 0.08333 FLOPs/byte, which is far below the ridge point. I therefore classified it as memory-bound.

The matrix multiplication has an estimated intensity of 1365.33 FLOPs/byte, which is far above the ridge point. I therefore classified it as compute-bound.

The execution and retained summary for Part C are shown in Fig. 16.

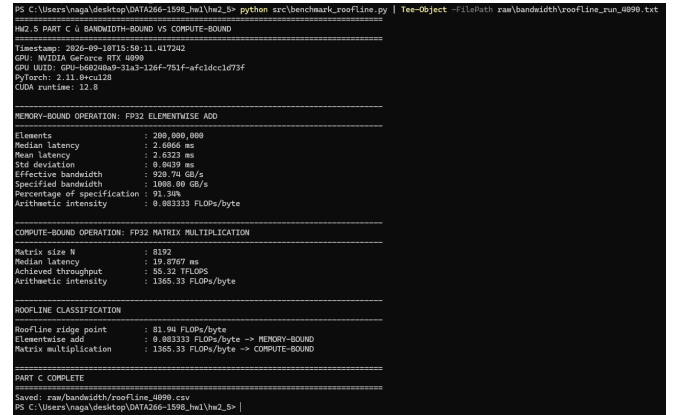


Fig. 16. Execution of the memory-bound and compute-bound tests.

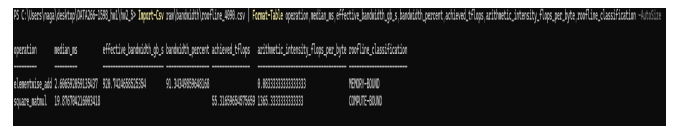


Fig. 16 (cont.). Saved bandwidth, arithmetic-intensity, and roofline summary.

TABLE VII. PART C ROOFLINE MEASUREMENTS

Operation	Problem Size	Median (ms)	Performance	Intensity	Roofline Side
FP32 elementwise add	200,000,000 elements	2.6066	920.74 GB/s	0.08333 FLOPs/byte	Memory-bound
FP32 square matmul	8192×8192	19.8767	55.32 TFLOPS	1365.33 FLOPs/byte	Compute-bound

This was useful because it showed me that one GPU does not have one single performance limit. The limit depends on the kind of work being performed.

V. PART D: THE COST OF ATTENTION

A. Why Sequence Length Matters

This was the most important part of the assignment for understanding what comes later in Transformers.

The direct scaled dot-product-attention equation is

$$Attention(Q,K,V) = softmax(QK^T/\sqrt{d_h}) V \quad (15)$$

I used a batch size of 4, eight attention heads, a head dimension of 64, and BF16 precision.

The Query, Key, and Value tensors therefore have the general shape

$$[B, H, L, d_h] \quad (16)$$

When Query is multiplied by Key transpose, the score tensor becomes

$$[B, H, L, L] \quad (17)$$

The sequence length appears twice. This is the reason I expected the memory requirement for the explicitly materialized score and probability matrices to contain an L^2 term.

B. Naive Attention Implementation

I implemented the direct form instead of calling an optimized attention kernel. The main operations were equivalent to:

```
scores = torch.matmul(
    q,
    k.transpose(-2, -1)
)

scores = scores / math.sqrt(head_dim)

probabilities = torch.softmax(
    scores,
    dim=-1
)

output = torch.matmul(
    probabilities,
    v
)
```

The script preparation and coarse-sweep execution are shown in Fig. 17.

```
PS C:\Users\naga\desktop\DATA266-1598_hw1\hw2_5> Get-Item src\benchmark_attention.py

Directory: C:\Users\naga\desktop\DATA266-1598_hw1\hw2_5\src

Mode                LastWriteTime         Length Name
----                -
-a-----          9/10/2026   4:00 PM           10147 benchmark_attention.py
```

Fig. 17. Naive attention benchmark script prepared for the RTX 4090.

[illegible]

Fig. 17 (cont.). Required coarse sequence-length sweep from 512 through 16,384.

The retained result table from that run is shown in Fig. 18.

```
PS C:\Users\naga\Desktop\DATA266-1598_hwi\hwi2_S> Import-Csv path\raw\attention\attention_naive_coarse_4990.csv |
    Format-Table sequence_length peak_allocated_gib peak_reserved_gib median_latency_ms status -AutoSize

sequence_length peak_allocated_gib peak_reserved_gib median_latency_ms status
-----
512      0.0646979703125      0.050701875      0.110752061407/65227      SUCCESS
1024     0.1485598703125      0.1646025     0.438271199395913086      SUCCESS
2048     0.5391845703125     0.55859375     1.679040082322876      SUCCESS
4096     2.0704345703125     2.05283125     7.356141295848333      SUCCESS
8192     8.1129345703125     8.14051125     28.97837611609336      SUCCESS
16384    32.2579345703125    32.26953125    102.2235107421875      SUCCESS

PS C:\Users\naga\Desktop\DATA266-1598_hwi\hwi2_S>
```

Fig. 18. Saved naive-attention coarse-sweep measurements.

C. Required Sequence-Length Measurements

The required sequence lengths were

$$L \in \{512, 1024, 2048, 4096, 8192, 16384\} \quad (18)$$

The main naive measurements are summarized in Table VIII.

TABLE VIII. NAIVE ATTENTION MEASUREMENTS

L	Peak GiB	Median Forward (ms)
512	0.0470	0.1108
1024	0.1486	0.4383
2048	0.5392	1.8790
4096	2.0704	7.3564
8192	8.1329	28.9782
16384	32.2579	892.2235

The change from 8192 to 16,384 was especially interesting. PyTorch reported the allocator peak increasing from about 8.13 GiB to 32.26 GiB. The measured latency also jumped from about 28.98 ms to 892.22 ms.

Because the RTX 4090 does not physically contain 32 GiB of VRAM, I did not accept the allocator number without checking it.

D. Separate 16K Verification

I repeated the 16,384-token run and monitored physical GPU memory using nvidia-smi at the same time.

Figs. 19 and 20 show the evidence from this check.



Fig. 19. Separate verification of the 16,384-token naive-attention run.

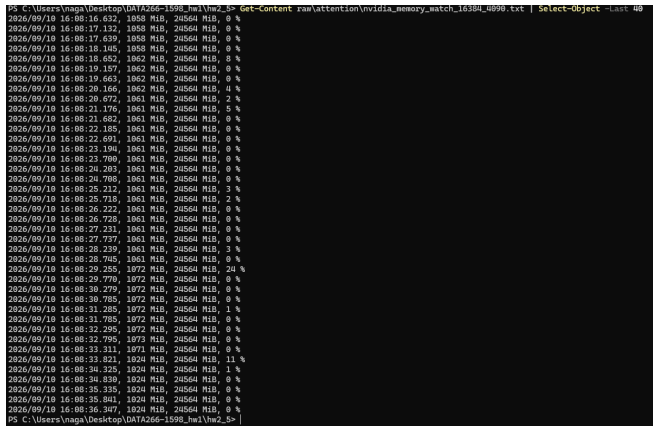


Fig. 19 (cont.). Tail of the physical-memory monitor during the 16K experiment.

The highest physical device-memory sample was approximately 23,945 MiB out of 24,564 MiB total.

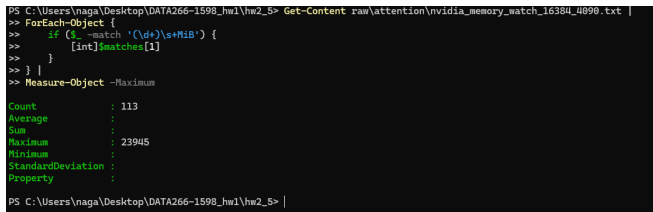


Fig. 20. Peak physical GPU-memory value extracted from the monitor log.

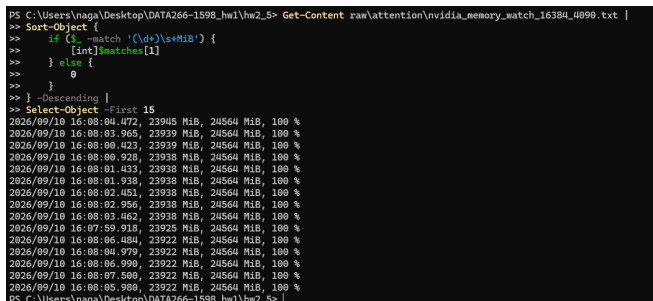


Fig. 20 (cont.). Highest physical-memory samples observed during the large-sequence run.

This helped me understand that PyTorch's allocator-reported value and the physical VRAM shown by nvidia-smi were not the same thing in this Windows WDDM environment. The process was able to continue through memory oversubscription or paging instead of immediately returning a clean CUDA out-of-memory error.

E. Naive OOM Boundary Search

After the coarse sweep, I tested longer sequences to search for the failure boundary.

The first progression was 17,408, 17,920, 18,432, and 18,944.

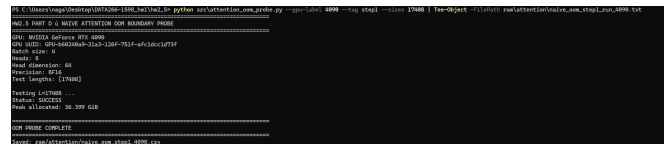


Fig. 21. Naive boundary probe at L=17,408.

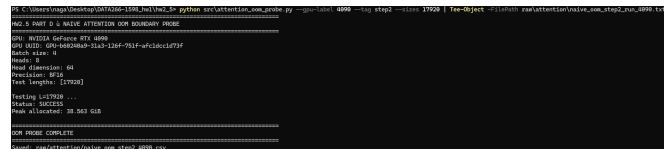


Fig. 21 (cont.). Naive boundary probe at L=17,920.

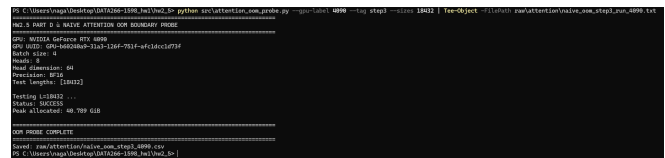


Fig. 22. Naive boundary probe at L=18,432.

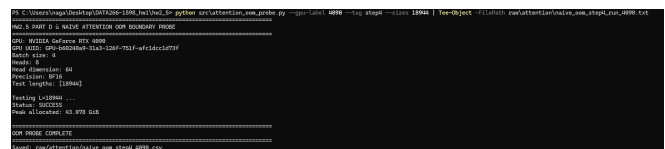


Fig. 22 (cont.). Largest safely tested naive probe at L=18,944.

The largest safely tested successful naive sequence was therefore

$$L_{max,naive} = 18,944 \quad (19)$$

I did not observe a clean smallest failing sequence.

I also performed additional capped checks around the physical-memory region.

Fig. 23. Additional capped test at L=13,312.

Fig. 23 (cont.). Additional capped test at L=13,824.

Fig. 24. Additional capped naive-attention test at L=14,080.

These checks supported the same conclusion: in the tested WDDM environment, the expected clean CUDA OOM did not appear before memory oversubscription became the main issue.

I therefore report the actual experimental result rather than inventing an exact failure boundary:

Largest safely tested naive success: 18,944.

Smallest clean failure: not observed.

F. Quadratic Memory Fit

The assignment asked me to confirm the quadratic term from the measured data.

I fitted the peak-memory measurements using

$$M(L) = aL^2 + bL + c \quad (20)$$

The measured quadratic coefficient was approximately

$$a = 1.1920929 \times 10^{-7} \text{ GiB/token}^2 \quad (21)$$

with a fit of approximately

$$R^2 = 1.0000 \quad (22)$$

The terminal fit output and the retained curve are shown in Fig. 25.

Fig. 25. Terminal output from the quadratic memory-fit calculation.

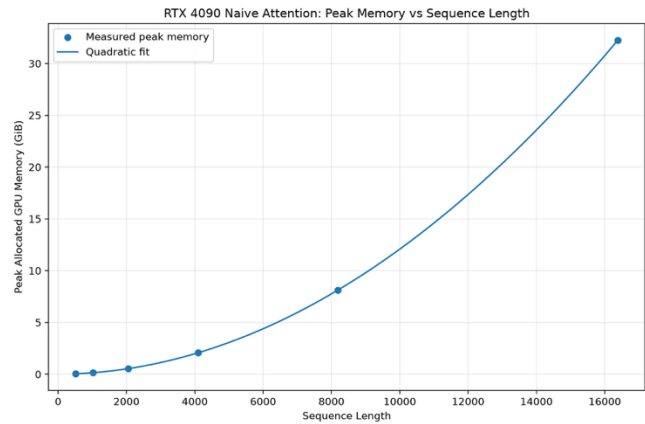


Fig. 25 (cont.). Retained naive-attention peak-memory curve.

The clean scientific plot is shown in Fig. 26.

The coefficient also makes sense from the tensors in the code. With batch size 4 and eight heads, two major BF16 L×L intermediates require approximately

$$4 \times 8 \times 2 \times 2 = 128 \text{ bytes} \quad (23)$$

for every L^2 term. Converting 128 bytes to GiB gives

$$128/2^{30} \approx 1.192 \times 10^{-7} \text{ GiB} \quad (24)$$

which is almost exactly the fitted coefficient. This was a useful check because the measured curve matched what I expected from the actual intermediate tensors.

G. First Optimized Attempt: Flash Attention

My first optimized attempt explicitly forced PyTorch's Flash Attention backend.

I expected the optimized backend to avoid materializing the complete attention matrix in the same way as my naive code. However, the run did not succeed.

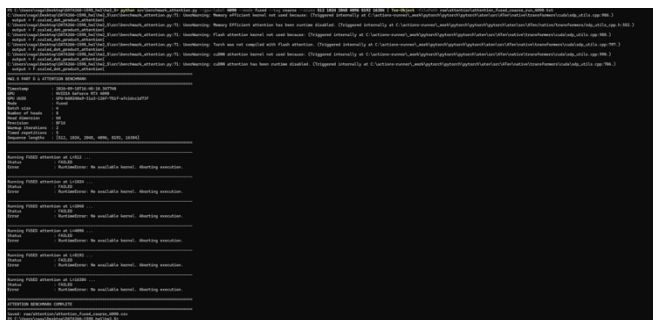


Fig. 26. First coarse optimized-attention attempt using the forced Flash backend.



Fig. 27 (cont.). Required sequence-length tests reporting failure.

The saved error showed

`RuntimeError: No available kernel.`

Instead of removing this failed run, I kept it because it was an actual finding about the software stack.

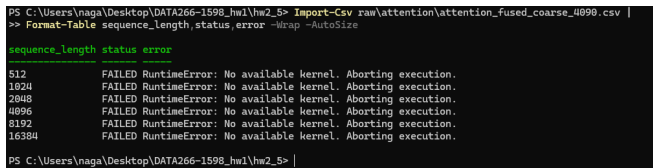


Fig. 28. Saved error output from the failed Flash Attention run.

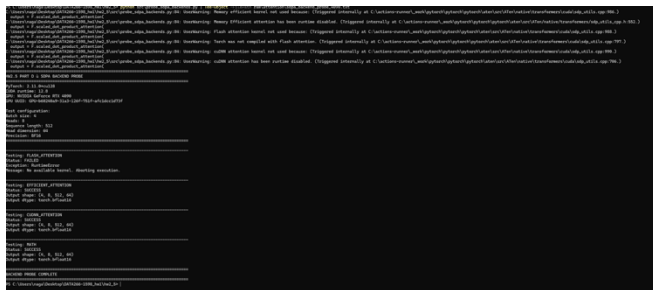


Fig. 28 (cont.). Backend probe used to determine which PyTorch SDPA implementations were available.

The backend probe showed that Flash Attention failed in the tested environment, while Efficient Attention, cuDNN Attention, and the Math backend were available.

Because the assignment allows a fused or memory-efficient implementation, I explicitly selected Efficient Attention for the corrected experiment instead of allowing an unknown fallback.

H. Efficient Attention Measurements

The corrected experiment used the same batch size, number of heads, head dimension, BF16 precision, and required sequence lengths as the naive experiment.

The successful coarse run is shown in Fig. 29.

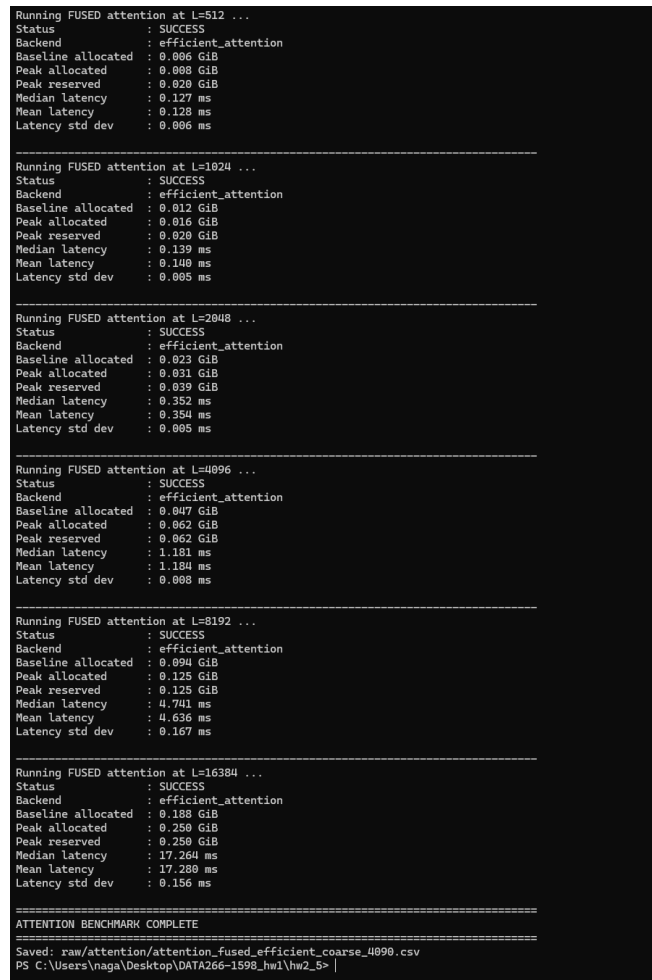


Fig. 29. Corrected Efficient Attention coarse benchmark.

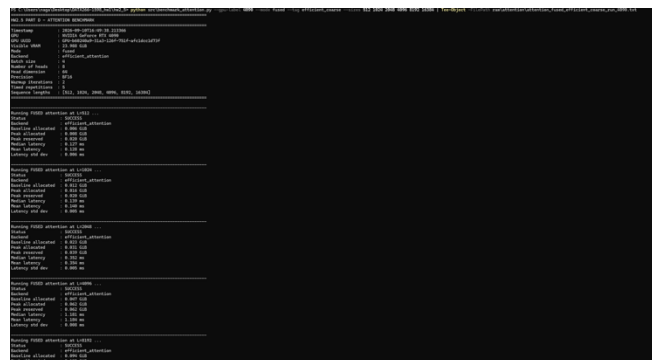


Fig. 29 (cont.). Retained output from the successful efficient-attention sweep.

Table IX compares the two implementations.

TABLE IX. NAIVE AND EFFICIENT ATTENTION COMPARISON

L	Naive GiB	Naive ms	Efficient GiB	Efficient ms	Speedup	Mem. Reduction
512	0.0470	0.1108	0.0078	0.1267	0.87×	6.02×
1024	0.1486	0.4383	0.0156	0.1393	3.15×	9.51×
2048	0.5392	1.8790	0.0313	0.3523	5.33×	17.25×
4096	2.0704	7.3564	0.0625	1.1807	6.23×	33.13×
8192	8.1329	28.9782	0.1250	4.7410	6.11×	65.06×
16384	32.2579	892.2235	0.2500	17.2636	51.68×	129.03×

At L=512, the optimized implementation was actually slightly slower. The workload was so small that the optimized path did not yet provide a latency advantage.

By L=4096 and L=8192, the difference was much clearer. At 8192, the naive implementation took about 28.98 ms while Efficient Attention took about 4.74 ms, giving a measured speedup of about 6.11 $\times$.

At 16,384, the measured speedup increased to 51.68 $\times$. I kept this number because it is the real end-to-end measurement, but I do not interpret it as a pure kernel speedup. The naive 16K run was already affected by memory oversubscription and likely paging.

The retained speedup table and plot are shown in Fig. 30.

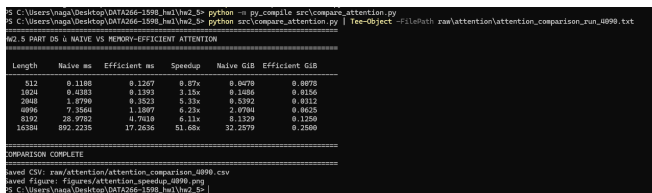


Fig. 30. Saved naive-versus-efficient speedup table.

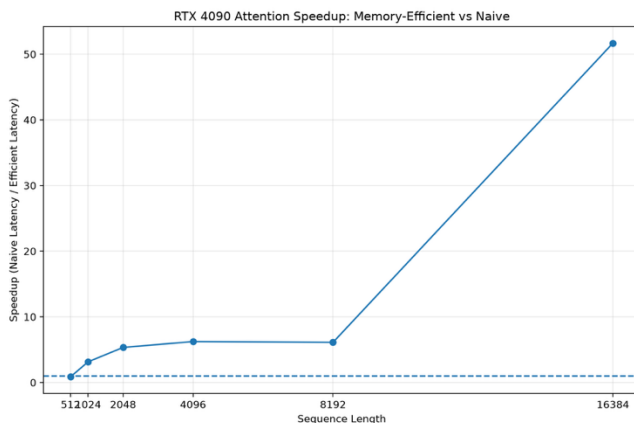


Fig. 30 (cont.). Retained attention speedup visualization.

The clean speedup plot saved under figures/ is shown in Fig. 31.

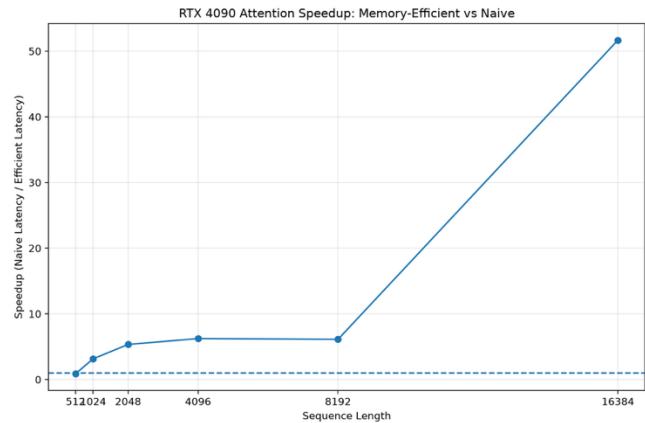


Fig. 31. Clean attention speedup figure generated from the retained benchmark results.

I. Efficient Attention Boundary Search

The optimized implementation allowed much larger sequence lengths, so I continued the boundary search.

The first large-sequence search and its saved table are shown in Fig. 32.

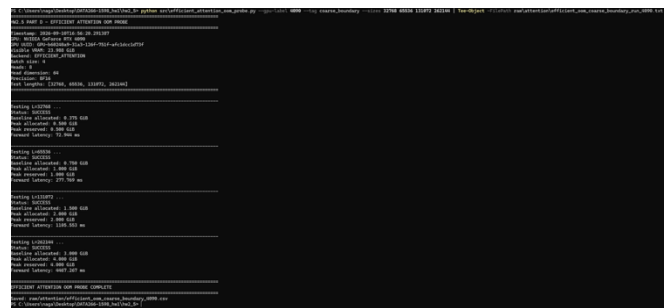


Fig. 32. Coarse Efficient Attention large-sequence search.

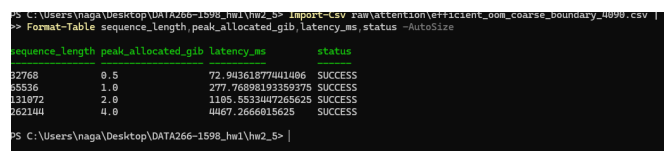


Fig. 32 (cont.). Saved coarse boundary-search table.

I then extended the search further.

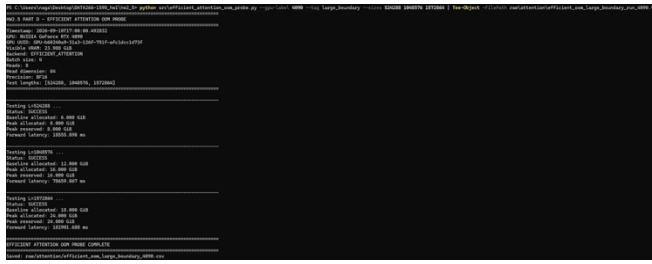


Fig. 33. Extended Efficient Attention large-sequence search.

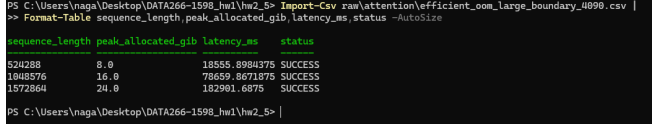


Fig. 33 (cont.). Saved table from the extended search.

Selected successful measurements are shown in Table X.

TABLE X. SELECTED EFFICIENT ATTENTION LARGE-SEQUENCE RESULTS

L	Peak GiB	Forward (ms)
32,768	0.5	72.94
65,536	1.0	277.77
131,072	2.0	1105.55
262,144	4.0	4467.27
524,288	8.0	18555.90
1,048,576	16.0	78659.87
1,572,864	24.0	182901.69

The largest safely tested successful sequence was

$$L_{max,efficient} = 1,572,864 \quad (25)$$

Again, I did not observe a clean smallest failing sequence before reaching the safely chosen stopping point.

My final boundary report is therefore:

Naive largest safe success: 18,944

Efficient largest safe success: 1,572,864

Clean smallest failure: not observed

J. What the Optimized Kernel Avoids

The naive implementation explicitly stores the complete $L \times L$ score matrix and then another large probability matrix after softmax. A memory-efficient implementation reorganizes the calculation so those full intermediates do not need to remain in GPU global memory in the same way. The work is handled in smaller blocks, which reduces temporary storage and global-memory traffic. This is why the memory requirement can be dramatically smaller even though the mathematical attention operation remains the same [6].

VI. PART E: SUSTAINED LOAD AND THERMAL BEHAVIOR

A. Why I Needed a Long Run

The earlier benchmarks were short. They were useful for measuring peak throughput, but they did not show what happens after the GPU has been under heavy load for many minutes.

For this reason, I ran repeated BF16 8192×8192 matrix multiplication for approximately 20 minutes.

During the test, telemetry was sampled about every five seconds. The logged fields included GPU clock, memory clock, temperature, power draw, power limit, utilization, and memory usage.

The UUID-labelled run ID was E-4090-SUSTAINED.

B. Sustained Workload Execution

The running and completed workload are shown in Fig. 34.

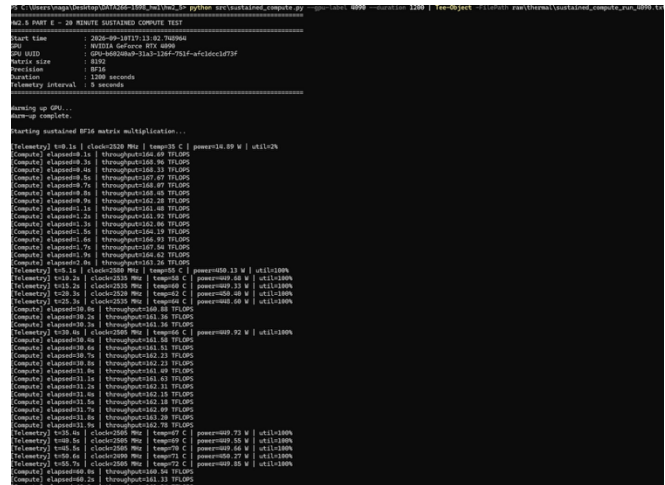


Fig. 34. Sustained BF16 compute workload while it was running.

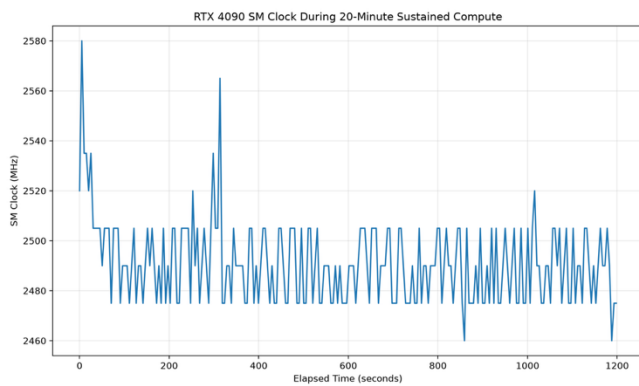


Fig. 36. Retained GPU clock plot from the sustained experiment.

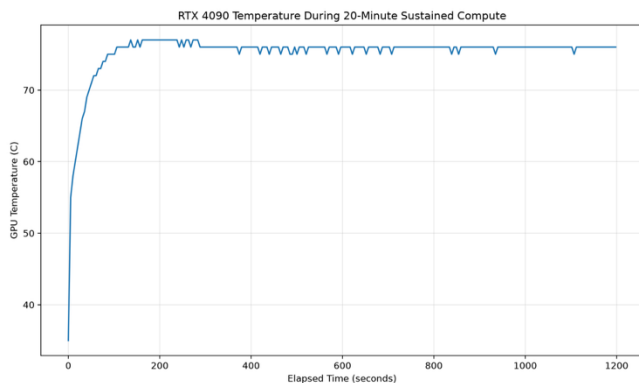


Fig. 36 (cont.). Retained GPU temperature plot from the sustained experiment.

The combined retained figure is shown in Fig. 37.

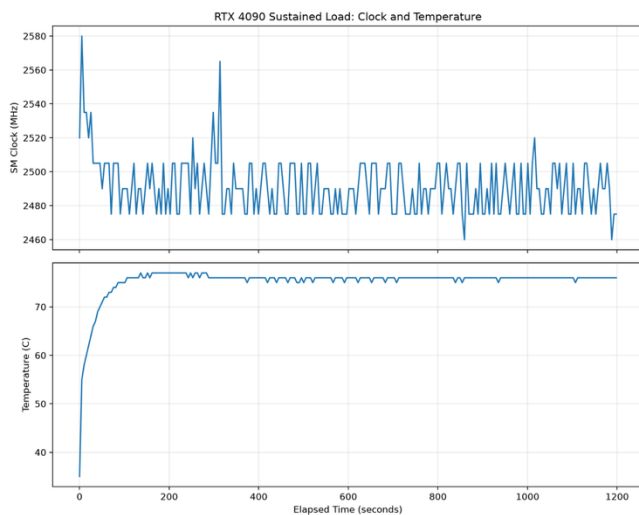


Fig. 37. Retained combined clock-and-temperature plot from the sustained experiment.

The clean clock, temperature, and combined figures from the figures/ directory are shown in Figs. 38 and 39.

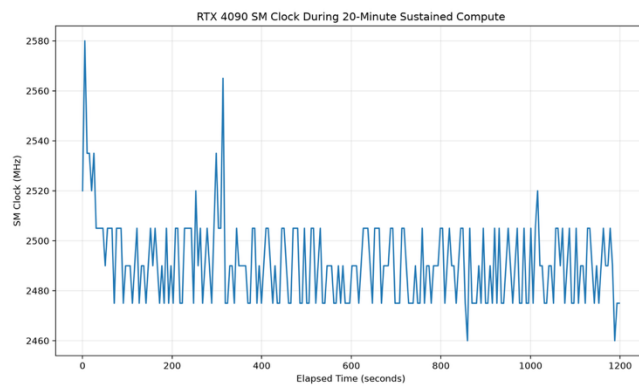


Fig. 38. Clean SM-clock plot generated from the thermal log.

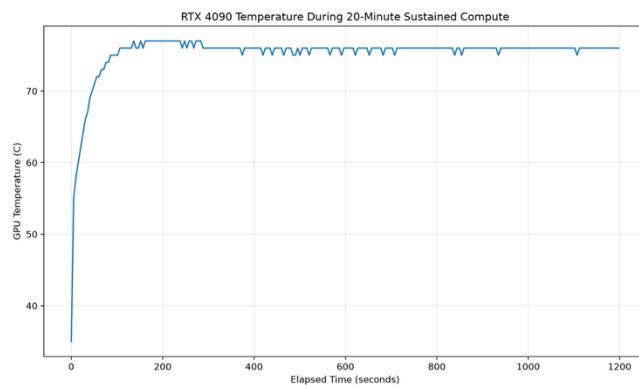


Fig. 38 (cont.). Clean temperature plot generated from the thermal log.

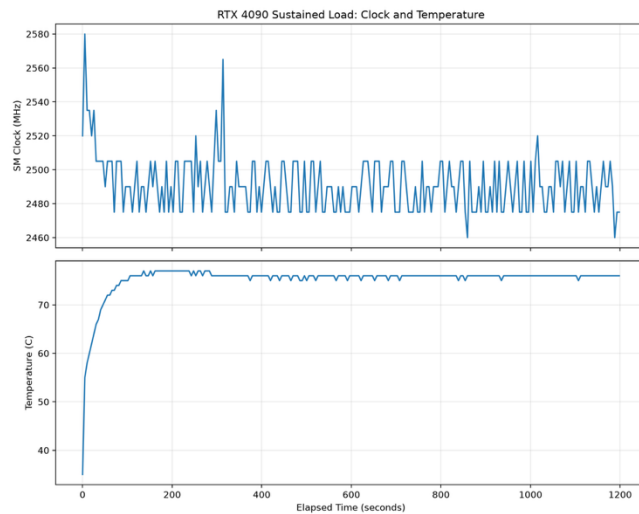


Fig. 39. GPU SM clock and temperature versus time during the sustained BF16 workload.

The GPU warmed from its initial cool state and then settled around the mid-70s Celsius.

The average SM clock decreased from about 2537.5 MHz during the first 30 seconds to about 2488 MHz during the final five minutes. That is a relatively small change.

E. Throttling Analysis

I did not observe a clear severe thermal-throttling event.

The maximum temperature was 77 degrees Celsius, while the final five-minute average power was 449.55 W against a configured 450 W power limit.

The clock did not show a sudden temperature-related collapse, and the final five-minute throughput still remained at 94.76% of the early peak.

The strongest limit visible in my measurements was therefore the power ceiling rather than an obvious thermal throttle.

For the assignment field asking for throttle onset, I report

No clear thermal-throttle onset observed.

VII. PART F: SUMMARY OF REQUIRED MEASUREMENTS

Table XII is the required HW2.5.1 summary. I include the corresponding run ID in the notes so that each result can be traced to the UUID-labelled experiment in RUN_LOG.txt.

TABLE XII. TABLE HW2.5.1 — SUMMARY OF RTX 4090 MEASUREMENTS

Measurement	Your GPU	Notes
Peak achieved TFLOPS (BF16)	164.69 TFLOPS	N=8192; run B-4090-MATMUL.
% of theoretical peak (BF16)	99.69%	164.6871 TFLOPS vs. dense 165.2 TFLOPS reference; run B-4090-MATMUL.
Effective bandwidth (GB/s)	920.74 GB/s	91.34% of the specified 1008 GB/s; run C-4090-ROOFLINE.
Naive attention OOM length	No clean OOM; largest safe success = 18,944	WDDM oversubscription prevented a clean CUDA OOM in the safely tested range; run D-4090-NAIVE.
Fused/memory-efficient attention OOM length	No clean OOM; largest safe success = 1,572,864	Efficient Attention reached ~24 GiB peak allocation at the largest safely tested length; run D-4090-EFFICIENT-OOM.
Steady-state / peak throughput	94.76%	160.11 TFLOPS final-5-min avg ÷ 168.96 TFLOPS first-30-s peak; run E-4090-SUSTAINED.
Throttle onset	None clearly observed	Max temp 77 °C; final avg power 449.55 W vs. 450 W limit; run E-4090-SUSTAINED.

Every measurement in this table corresponds to the RTX 4090 identified by UUID

GPU-b60240a9-31a3-126f-751f-afc1dcc1d73f

VIII. FAILURE AND ERROR ANALYSIS

The unsuccessful experiments were important because they showed me where the software and operating environment affected the hardware tests.

A. Flash Attention Was Not Available

My first optimized-attention attempt forced the Flash Attention backend. The actual GPU run returned “No available kernel.”

Instead of deleting the failure, I tested the available PyTorch SDPA backends individually. Efficient Attention and cuDNN Attention worked, while the forced Flash backend did not.

I therefore repeated the assignment using the explicitly available Efficient Attention backend.

This was a better result than silently allowing PyTorch to choose an unknown fallback because I could state exactly which backend produced the final measurements.

B. A Clean OOM Boundary Was Not Observed

I expected the naive attention search eventually to end with a normal CUDA out-of-memory error.

Instead, PyTorch's reported allocation exceeded physical VRAM while nvidia-smi showed the device itself close to full memory.

The Windows WDDM environment continued running through memory oversubscription or paging.

For this reason, forcing the test further would not have given a clean device-only VRAM boundary. It also could have made the workstation unstable.

I therefore report the largest safely tested successful values instead of creating a failure length that I never measured.

C. Values Above the Nominal Theoretical Reference

Several short matrix benchmarks slightly exceeded the fixed reference used in the percentage calculation.

I kept those values unchanged.

A fixed theoretical reference and a dynamically boosted consumer GPU do not represent exactly the same operating condition, and short timing measurements also contain normal variation.

D. The 16K Speedup Needs Context

The $51.68\times$ measured speedup at 16K is a real end-to-end result, but the naive side was already affected by memory oversubscription.

For a cleaner fully resident comparison, the 8192-token experiment is more useful. At that point the naive implementation used approximately 8.13 GiB and the measured speedup was about $6.11\times$.

IX. REPRODUCIBILITY AND DELIVERABLE TRACEABILITY

I kept the raw measurements and logs instead of reporting values only inside this paper.

TABLE XIII. HW2.5 DELIVERABLE TRACEABILITY

Deliverable / Evidence	Repository Location
Complete nvidia-smi -q	raw/gpu_info/nvidia_smi_4090.txt
GPU identity and UUID	raw/gpu_info/gpu_identity_4090.txt
Vendor specification notes	raw/gpu_info/vendor_specs_4090.txt
Reservation record	raw/reservations/
GPU-hour record	GPU_HOURS.md
Matrix benchmark	raw/matmul/
FP8 benchmark	raw/matmul/fp8_benchmark_4090.csv
Bandwidth / roofline experiment	raw/bandwidth/
Naive attention results	raw/attention/attention_naive_coarse_4090.csv
Efficient attention results	raw/attention/attention_fused_efficient_coarse_4090.csv
Attention backend probe	raw/attention/sdpa_backend_probe_4090.txt
Thermal telemetry	raw/thermal/thermal_4090.csv
Sustained throughput	raw/thermal/throughput_4090.csv
Summary metrics	METRICS.md
UUID-labelled run trace	RUN_LOG.txt
AI-use record	AI_USE.md
Figures	figures/
Execution screenshots	raw/screenshots/4090/

The assignment also requires the repository tag hw2-5. This should be verified in the original Git repository before final submission.

X. DISCUSSION

This assignment changed the way I think about GPU specifications.

The matrix experiment showed me that theoretical compute capability is only useful when the workload is large enough to use the GPU well. At $N=1024$, the card

was powerful, but the workload was simply too small to reach the same throughput seen at the larger matrix sizes.

The roofline experiment showed a different limitation. The elementwise addition does almost no arithmetic compared with the amount of data it moves. Even a card with very high compute capability cannot turn that operation into a compute-heavy workload.

The attention experiment made the sequence-length problem much more concrete. I could actually see memory rise from a very small value at 512 tokens to several GiB at 8192 and then into memory-oversubscription behavior at 16,384.

The quadratic fit was especially useful because the measured coefficient agreed with the memory expected from the large BF16 $L \times L$ intermediate tensors in my implementation. That connected the complexity expression to the tensors I actually created.

The optimized-attention experiment also showed me that implementation details matter almost as much as the equation itself. The mathematical operation remained attention, but changing how the intermediate data was handled reduced the memory requirement dramatically.

Finally, the sustained run showed why short peak numbers should be interpreted carefully. The card remained fast after warming up, but it did so while operating very close to the 450 W power ceiling.

XI. LIMITATIONS

All measured results in this report came from one assigned RTX 4090 workstation. They should therefore be treated as measurements of this specific hardware and software environment rather than universal performance guarantees for every RTX 4090.

The Windows WDDM memory model also affected the attention OOM experiment. Because memory oversubscription occurred, I could not obtain a clean smallest failing CUDA sequence length within the safely tested range.

The forced Flash Attention backend was unavailable in the installed PyTorch environment. I therefore used the explicitly available Efficient Attention backend for the optimized comparison.

Some matrix results also depend on GPU boost clocks and very short timing intervals. Small variation should therefore be expected if the same experiment is repeated.

The 16K naive-attention latency was affected by memory paging, so its $51.68\times$ optimized speedup should be interpreted as an end-to-end workstation result rather than a pure GPU-kernel comparison.

XII. CONCLUSION

Before this assignment, I mostly thought about a GPU using values such as TFLOPS, VRAM, and memory bandwidth from the specification sheet. After completing these experiments, I understood that real performance also depends on workload size, precision, arithmetic intensity, implementation, software support, power, and temperature.

The best required BF16 matrix result in my experiment was 164.69 TFLOPS at $N=8192$, or 99.69% of the dense reference used in the report. The memory-bound elementwise test reached 920.74 GB/s, which was 91.34% of the specified memory bandwidth.

The attention experiments showed the strongest algorithmic effect. Naive attention followed the expected quadratic memory trend, and the measured coefficient matched what I expected from the explicit $L \times L$ BF16 intermediate tensors. Memory-efficient attention reduced the reported memory requirement dramatically and allowed much larger tested sequences.

The failed Flash Attention experiment was also useful because it showed me that hardware capability alone does not guarantee that a particular software kernel will be available.

Finally, the 20-minute workload showed that the RTX 4090 maintained 94.76% of its early peak throughput during the final five minutes while operating close to its 450 W power limit.

Overall, the assignment gave me a much more practical understanding of what matrix multiplication, memory bandwidth, and attention actually cost on a real GPU. That makes the later Transformer material easier to understand because the computational cost is no longer only a formula to me.

REFERENCES

- [1] NVIDIA Corporation, “NVIDIA Ada GPU Architecture,” NVIDIA, 2022. [Online]. Available: <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>
- [2] NVIDIA Corporation, “GeForce RTX 4090 Graphics Card,” NVIDIA. [Online]. Available: <https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/>
- [3] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] PyTorch, “Scaled dot product attention,” PyTorch Documentation. [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html
- [6] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems*, 2022.